Lab Report No 3
Solving a regression problem
Artificial Intelligence

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

11th Mar,2025

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1(i):

Solution:

Brief description (3-5 lines)

Simple linear regression:

Simple linear regression is an approach for predicting a response using a single feature. It is assumed that the two variables are linearly related. Hence, we try to find a linear function that predicts the response value(y) as accurately as possible as a function of the feature or independent variable(x).

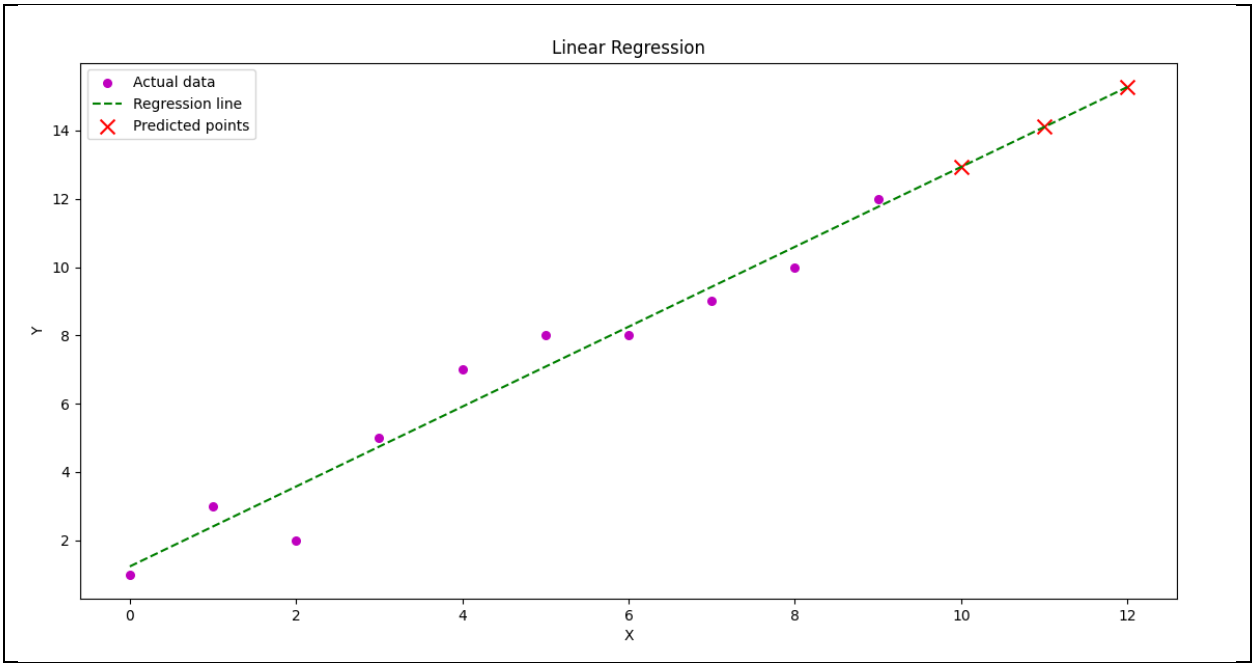
The code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
# Function to estimate regression coefficients
def estimate_coef(x, y):
    n = np.size(x)  # Number of data points
    m_x, m_y = np.mean(x), np.mean(y)  # Mean of x and y
    # Cross-deviation and deviation about x
    SS_xy = np.sum(y * x) - n * m_y * m_x
    SS_xx = np.sum(x * x) - n * m_x * m_x
    # Regression coefficients
    b_1 = SS_xy / SS_xx
    b_0 = m_y - b_1 * m_x
    return (b_0, b_1)
# Function to predict new data points
def predict_new_values(x_new, b):
    return b[0] + b[1] * x_new  # Regression formula Y = b0 + b1*X
# Function to evaluate regression performance
def evaluate_regression(y_actual, y_predicted):
    mse = mean_squared_error(y_actual, y_predicted)
    r2 = r2_score(y_actual, y_predicted)
    return mse, r2
# Function to plot regression line, original data, and new predictions
def plot_regression_line(x, y, b, x_new=None, y_new=None):
    # Scatter plot for actual data
    plt.scatter(x, y, color="m", marker="o", s=30, label="Actual data")
    # Extend regression line to cover new X values
    x_extended = np.append(x, x_new) if x_new is not None else x
    y_pred_extended = b[0] + b[1] * x_extended
    plt.plot(x_extended, y_pred_extended, color="g", linestyle="--", label="Regression line")
    # Plot predicted new data points if provided
    if x_new is not None and y_new is not None:
        plt.scatter(x_new, y_new, color="r", marker="x", s=100, label="Predicted points")
```

```
    # Labels, legend, and title
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.title('Linear Regression')
    plt.legend()
    # Show plot
    plt.show()
# Main function
def main():
    # Training data
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])
    # Estimate coefficients
    b = estimate_coef(x, y)
    print(f"Regression Coefficients: b_0 = {b[0]:.2f}, b_1 = {b[1]:.2f}")
    # Predicting values for new data points
    x_new = np.array([10, 11, 12])  # New X values
    y_new = predict_new_values(x_new, b)
    # Printing predicted values
    print("Predicted values for new X points:")
    for i in range(len(x_new)):
        print(f"X = {x_new[i]}, Predicted Y = {y_new[i]:.2f}")
    # Compute predicted values for original dataset (for evaluation)
    y_pred_original = predict_new_values(x, b)
    # Evaluate regression performance
    mse, r2 = evaluate_regression(y, y_pred_original)
    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"R-squared Value (R2): {r2:.2f}")
    # Plot regression line with actual data and predicted new points
    plot_regression_line(x, y, b, x_new, y_new)
# Running the script
if __name__ == "__main__":
    main()
```

The results (Screenshot)

```
Regression Coefficients: b_0 = 1.24, b_1 = 1.17
Predicted values for new X points:
X = 10, Predicted Y = 12.93
X = 11, Predicted Y = 14.10
X = 12, Predicted Y = 15.27
Mean Squared Error (MSE): 0.56
R-squared Value (R2): 0.95
```



Lab Task No 1(ii):

Solution:

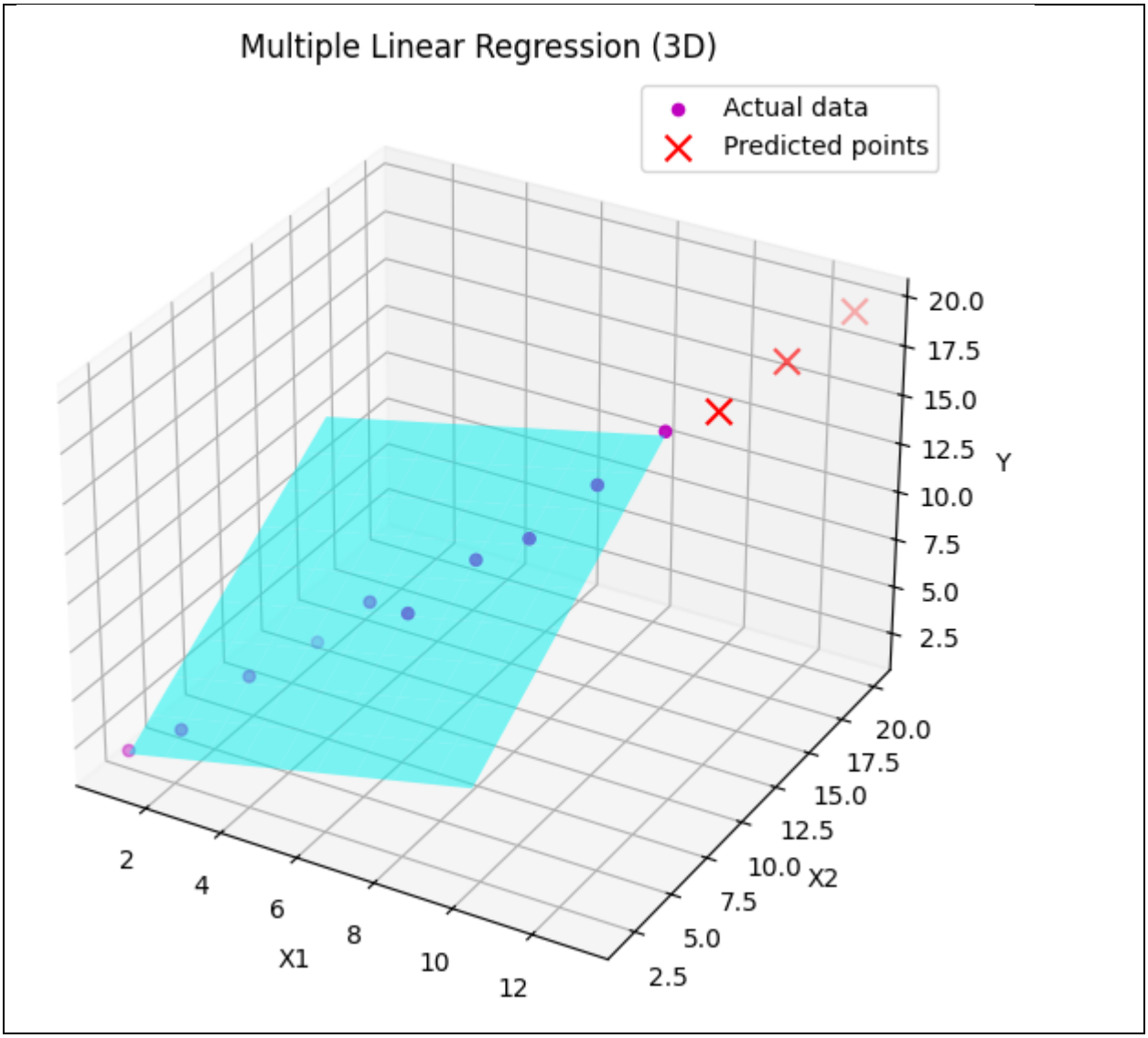
Brief description (3-5 lines)
Multiple Linear Regression: Multiple linear regression is a statistical technique that predicts the value of a dependent variable based on the values of two or more independent variables, extending simple linear regression to include multiple predictors.
The code

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Generate sample dataset (X: independent variables, y: dependent variable) 	
X = np.array([[1, 2], [2, 3], [3, 5], [4, 7], [5, 8],[6, 8], [7, 10], [8, 11], [9, 13], [10, 15]]) # Two independent variables
y = np.array([2, 3, 5, 6, 8, 8, 10, 11, 13, 15]) # Dependent variable
# Create and train the Multiple Linear Regression model 	
model = LinearRegression()
model.fit(X, y)
# Get regression coefficients 	
b_0 = model.intercept_ # Intercept (b_0)
b = model.coef_ # Coefficients (b_1, b_2, ...)
print(f"Regression Coefficients: b_0 = {b_0:.2f}, b_1 = {b[0]:.2f}, b_2 = {b[1]:.2f}")
# Predict values for the training set 	

```
y_pred = model.predict(X)
# Evaluate model performance
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared Value (R2): {r2:.2f}")
# Predict values for new data points
X_new = np.array([[11, 16], [12, 18], [13, 20]])  # New independent variable values
y_new_pred = model.predict(X_new)
print("Predicted values for new data points:")
for i in range(len(X_new)):
    print(f"X = {X_new[i]}, Predicted Y = {y_new_pred[i]:.2f}")
# 3D Plot of Multiple Linear Regression (for two independent variables)
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
# Scatter plot of actual data
ax.scatter(X[:, 0], X[:, 1], y, color='m', marker='o', label="Actual data")
# Create a surface plot for regression plane
X1_range = np.linspace(X[:, 0].min(), X[:, 0].max(), 10)
X2_range = np.linspace(X[:, 1].min(), X[:, 1].max(), 10)
X1_grid, X2_grid = np.meshgrid(X1_range, X2_range)
Y_pred_grid = b_0 + b[0] * X1_grid + b[1] * X2_grid
ax.plot_surface(X1_grid, X2_grid, Y_pred_grid, color='cyan', alpha=0.5)
# Plot predicted new data points
ax.scatter(X_new[:, 0], X_new[:, 1], y_new_pred, color='r', marker='x', s=100,
label="Predicted points")
# Labels and legend
ax.set_xlabel('X1')
ax.set_ylabel('X2')
ax.set_zlabel('Y')
ax.set_title('Multiple Linear Regression (3D)')
ax.legend()
# Show plot
plt.show()
```

The results (Screenshot)

```
Regression Coefficients: b_0 = 0.00, b_1 = 0.43, b_2 = 0.70
Mean Squared Error (MSE): 0.06
R-squared Value (R2): 1.00
Predicted values for new data points:
X = [11 16], Predicted Y = 15.92
X = [12 18], Predicted Y = 17.75
X = [13 20], Predicted Y = 19.57
```



Lab Task No 1(iii):

Solution:

Brief description (3-5 lines)
Polynomial Regression: Polynomial regression is a kind of linear regression in which the relationship shared between the dependent and independent variables Y and X is modeled as the nth degree of the polynomial. This is done to look for the best way of drawing a line using data points. Keep reading to know more about polynomial regression.
The code
import numpy as np import matplotlib.pyplot as plt

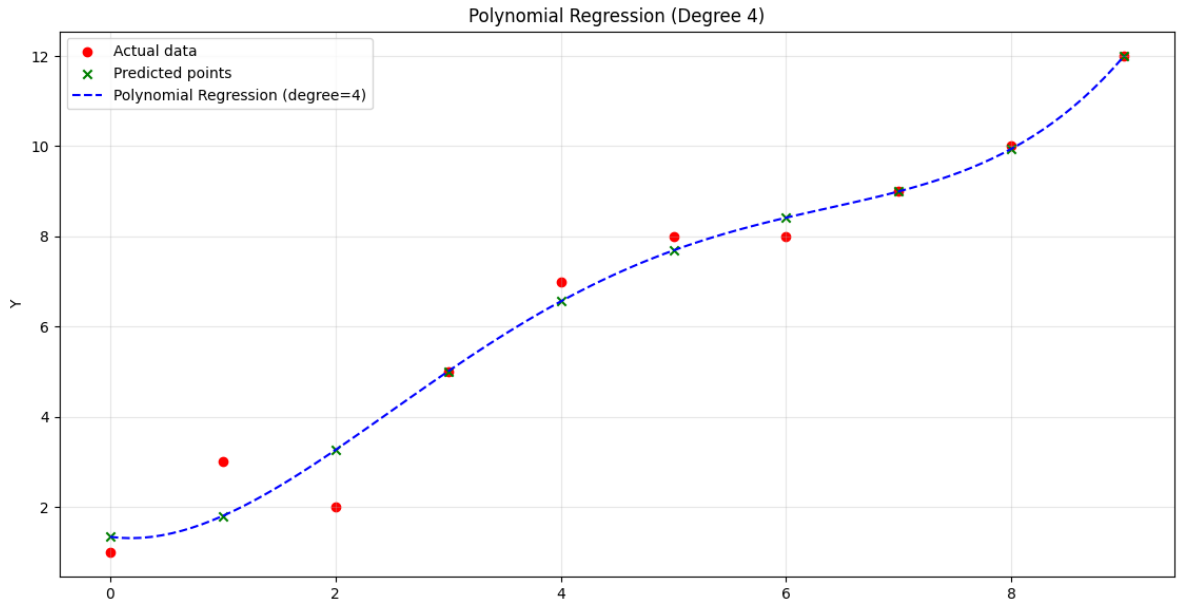
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
# Function to perform Polynomial Regression
def polynomial_regression(x, y, degree):
    # Reshaping x to a 2D array if it's not already
    if len(x.shape) == 1:
        x = x.reshape(-1, 1)
    # Transforming x into polynomial features
    poly = PolynomialFeatures(degree=degree)
    x_poly = poly.fit_transform(x)
    # Fitting the transformed features into Linear Regression model
    model = LinearRegression()
    model.fit(x_poly, y)
    # Predicting values
    y_pred = model.predict(x_poly)
    # Model Evaluation
    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)
    return model, poly, mse, r2, y_pred
# Function to plot Polynomial Regression curve
def plot_polynomial_regression(x, y, model, poly, degree, y_pred=None):
    # Create a new figure
    plt.figure(figsize=(10, 6))
    # Scatter plot of original data
    plt.scatter(x, y, color='red', marker='o', label="Actual data")
    # Optionally plot the predicted points
    if y_pred is not None:
        plt.scatter(x, y_pred, color='green', marker='x', label="Predicted points")
    # Generating smooth curve for polynomial regression
    x_smooth = np.linspace(min(x), max(x), 100).reshape(-1, 1)
    y_smooth = model.predict(poly.transform(x_smooth))
    # Plot the regression curve
    plt.plot(x_smooth, y_smooth, color='blue', linestyle='--', label=f'Polynomial Regression
(degree={degree})')
    # Labels and legend
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.title(f'Polynomial Regression (Degree {degree})')
    plt.legend()
    plt.grid(True, alpha=0.3)
    return plt
# Function to evaluate models with different polynomial degrees
def evaluate_polynomial_degrees(x, y, max_degree=5):
    results = []
```

```
    for degree in range(1, max_degree + 1):
        model, poly, mse, r2, y_pred = polynomial_regression(x, y, degree)
        results.append({
            'degree': degree,
            'model': model,
            'poly': poly,
            'mse': mse,
            'r2': r2,
            'y_pred': y_pred
        })
    return results
# Main function
def main():
    # Training Data
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])  # X values
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])  # Corresponding Y values
    # Optional: Split data into training and testing sets
    # x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
    # Try different polynomial degrees
    results = evaluate_polynomial_degrees(x, y, max_degree=4)
    # Print Results for each degree
    for result in results:
        degree = result['degree']
        mse = result['mse']
        r2 = result['r2']
        print(f"\nPolynomial Regression (Degree {degree})")
        print(f"Mean Squared Error (MSE): {mse:.4f}")
        print(f"R-squared Value (R²): {r2:.4f}")
    # Plot the regression curve for the best model (based on R²)
    best_model = max(results, key=lambda x: x['r2'])
    best_degree = best_model['degree']
    print(f"\nBest model: Polynomial Regression (Degree {best_degree})")
    plt = plot_polynomial_regression(
        x, y,
        best_model['model'],
        best_model['poly'],
        best_degree,
        best_model['y_pred']
    )
    # Display coefficient values for the best model
    model = best_model['model']
    print(f"\nIntercept: {model.intercept_:.4f}")
    print("Coefficients:", end=" ")
    for i, coef in enumerate(model.coef_):
        if i > 0:  # Skip the intercept term that PolynomialFeatures adds
            print(f"{coef:.4f}", end=" ")
```

```
    print()
    plt.show()
# Running the script
if __name__ == "__main__":
    main()
```

The results (Screenshot)

```
Polynomial Regression (Degree 1)
Mean Squared Error (MSE): 0.5624
R-squared Value (R²): 0.9525

Polynomial Regression (Degree 2)
Mean Squared Error (MSE): 0.5352
R-squared Value (R²): 0.9548

Polynomial Regression (Degree 3)
Mean Squared Error (MSE): 0.5329
R-squared Value (R²): 0.9550

Polynomial Regression (Degree 4)
Mean Squared Error (MSE): 0.3615
R-squared Value (R²): 0.9695

Best model: Polynomial Regression (Degree 4)
Warning: QT_DEVICE_PIXEL_RATIO is deprecated. Instead use:
   QT_AUTO_SCREEN_SCALE_FACTOR to enable platform plugin controlled per-screen factors.
   QT_SCREEN_SCALE_FACTORS to set per-screen DPI.
   QT_SCALE_FACTOR to set the application global scale factor.

Intercept: 1.3357
Coefficients: -0.3361 0.9706 -0.1808 0.0102
```



Lab Task No 1(iv):

Solution:

Brief description (3-5 lines)

Support Vector Regression:

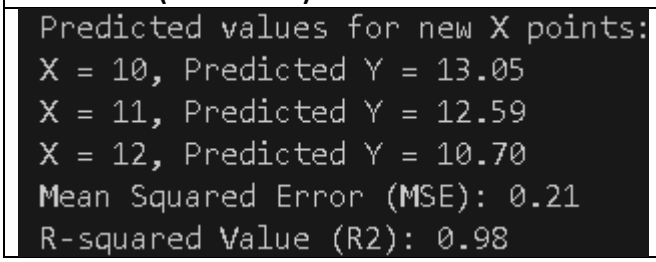
In machine learning, support vector machines are supervised max-margin models with associated learning algorithms that analyze data for classification and regression analysis.

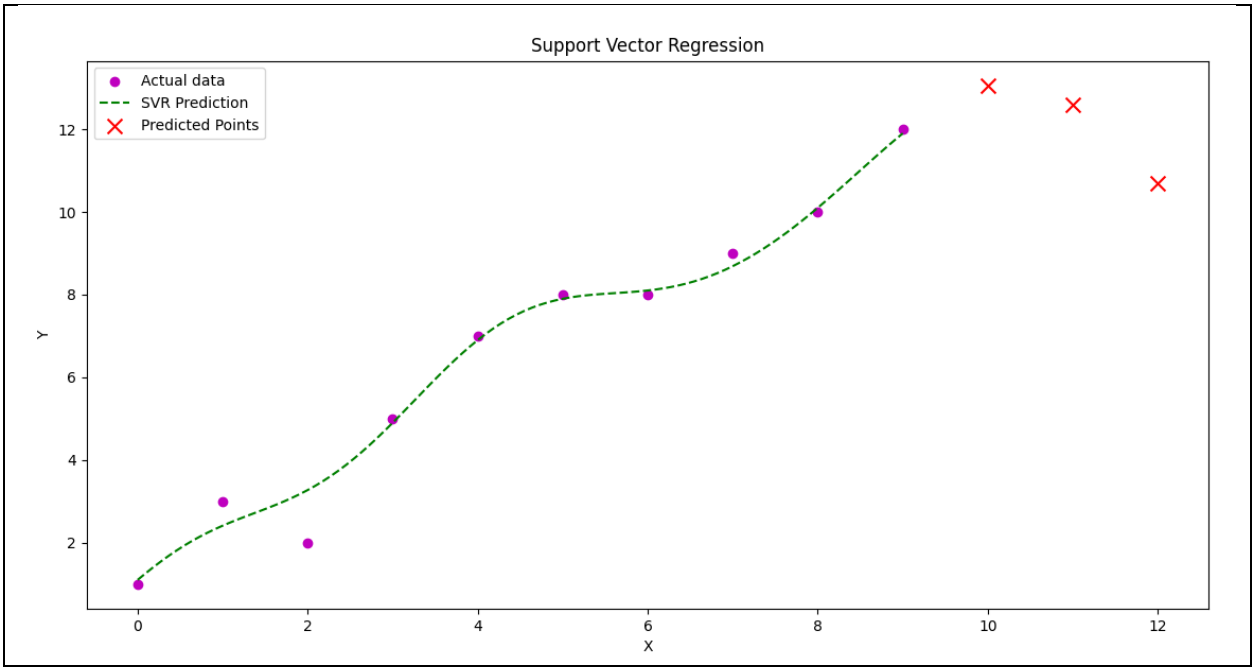
The code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
# Function to train Support Vector Regression model
def train_svr(x, y, kernel_type='rbf', C=1.0, epsilon=0.1):
    # Reshape x for SVR (SVR expects a 2D array)
    x = x.reshape(-1, 1)
    # Standardize the dataset
    scaler = StandardScaler()
    x_scaled = scaler.fit_transform(x)
    # Train SVR model
    svr_model = SVR(kernel=kernel_type, C=C, epsilon=epsilon)
    svr_model.fit(x_scaled, y)
    return svr_model, scaler
# Function to predict values using trained SVR model
def predict_svr(model, scaler, x_new):
    x_new = x_new.reshape(-1, 1)  # Reshape input for SVR
    x_new_scaled = scaler.transform(x_new)  # Standardize new inputs
    return model.predict(x_new_scaled)
# Function to evaluate SVR model
def evaluate_regression(y_actual, y_predicted):
    mse = mean_squared_error(y_actual, y_predicted)
    r2 = r2_score(y_actual, y_predicted)
    return mse, r2
# Function to plot SVR regression results
def plot_svr(x, y, model, scaler, x_new=None, y_new=None):
    plt.scatter(x, y, color="m", marker="o", label="Actual data")
    # Generate smooth curve for SVR prediction
    x_range = np.linspace(x.min(), x.max(), 100).reshape(-1, 1)
    x_range_scaled = scaler.transform(x_range)
    y_svr = model.predict(x_range_scaled)
    plt.plot(x_range, y_svr, color="g", linestyle="--", label="SVR Prediction")
    # Plot predicted new points
    if x_new is not None and y_new is not None:
```

```
        plt.scatter(x_new, y_new, color="r", marker="x", s=100, label="Predicted Points")
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.title('Support Vector Regression')
    plt.legend()
    plt.show()
# Main function
def main():
    # Training Data
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])
    # Train SVR Model
    svr_model, scaler = train_svr(x, y, kernel_type='rbf', C=100, epsilon=0.1)
    # Predict new values
    x_new = np.array([10, 11, 12])
    y_new = predict_svr(svr_model, scaler, x_new)
    # Print predicted values
    print("Predicted values for new X points:")
    for i in range(len(x_new)):
        print(f"X = {x_new[i]}, Predicted Y = {y_new[i]:.2f}")
    # Compute predictions for evaluation
    y_pred_original = predict_svr(svr_model, scaler, x)
    # Evaluate SVR Model
    mse, r2 = evaluate_regression(y, y_pred_original)
    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"R-squared Value (R2): {r2:.2f}")
    # Plot results
    plot_svr(x, y, svr_model, scaler, x_new, y_new)
# Run the script
if __name__ == "__main__":
    main()
```

The results (Screenshot)





Lab Task No 1(v):

Solution:

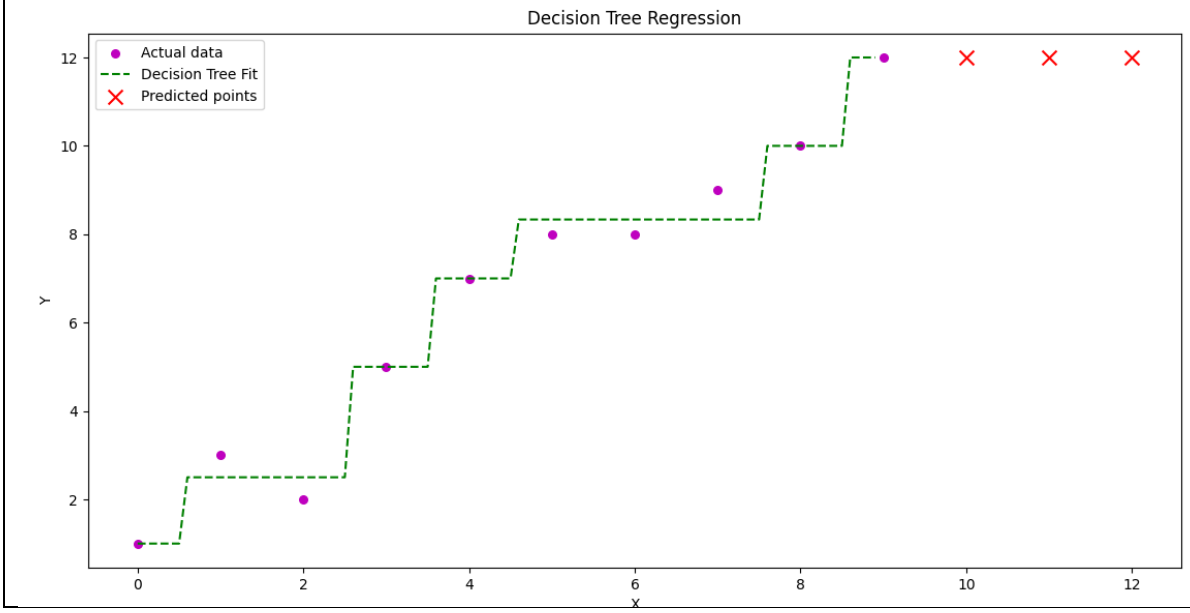
Brief description (3-5 lines)
Decision Tree Regression: A Decision Tree for regression is a model that predicts numerical values using a tree-like structure. It splits data based on key features, starting from a root question and branching out. Each node asks about a feature, dividing data further until reaching leaf nodes with final predictions.
The code

import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
# Function to train a Decision Tree Regressor 	
def train_decision_tree(x, y, max_depth=None):
  x = x.reshape(-1, 1) # Reshape for sklearn
  model = DecisionTreeRegressor(max_depth=max_depth, random_state=42)
  model.fit(x, y)
  return model
# Function to predict new values 	
def predict_values(model, x_new):
  x_new = x_new.reshape(-1, 1)
  return model.predict(x_new)
# Function to evaluate model performance 	

```
def evaluate_model(y_actual, y_predicted):
    mse = mean_squared_error(y_actual, y_predicted)
    r2 = r2_score(y_actual, y_predicted)
    return mse, r2
# Function to plot Decision Tree Regression results
def plot_decision_tree_regression(x, y, model, x_new=None, y_new=None):
    x_grid = np.arange(min(x), max(x), 0.1).reshape(-1, 1)  # High-resolution X
    y_pred_grid = model.predict(x_grid)  # Predictions for smooth curve
    # Plot actual data
    plt.scatter(x, y, color="m", marker="o", s=30, label="Actual data")
    # Plot Decision Tree Regression line
    plt.plot(x_grid, y_pred_grid, color="g", linestyle="--", label="Decision Tree Fit")
    # Plot new predictions if provided
    if x_new is not None and y_new is not None:
        plt.scatter(x_new, y_new, color="r", marker="x", s=100, label="Predicted points")
    # Labels, legend, and title
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.title('Decision Tree Regression')
    plt.legend()
    plt.show()
# Main function
def main():
    # Training data
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])
    # Train Decision Tree Regressor
    model = train_decision_tree(x, y, max_depth=3)
    # Predict new values
    x_new = np.array([10, 11, 12])  # New X values
    y_new = predict_values(model, x_new)
    # Print predicted values
    print("Predicted values for new X points:")
    for i in range(len(x_new)):
        print(f"X = {x_new[i]}, Predicted Y = {y_new[i]:.2f}")
    # Compute predictions for original dataset (for evaluation)
    y_pred_original = predict_values(model, x)
    # Evaluate model performance
    mse, r2 = evaluate_model(y, y_pred_original)
    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"R-squared Value (R2): {r2:.2f}")
    # Plot Decision Tree Regression results
    plot_decision_tree_regression(x, y, model, x_new, y_new)
# Running the script
if __name__ == "__main__":
    main()
```

The results (Screenshot)

```
Predicted values for new X points:
X = 10, Predicted Y = 12.00
X = 11, Predicted Y = 12.00
X = 12, Predicted Y = 12.00
Mean Squared Error (MSE): 0.12
R-squared Value (R2): 0.99
```



Lab Task No 1(vi):

Solution:

Brief description (3-5 lines)

Random Forest Regression:

Random forest regression is a supervised learning algorithm and bagging technique that uses an ensemble learning method for regression in machine learning. The trees in random forests run in parallel, meaning there is no interaction between these trees while building the trees.

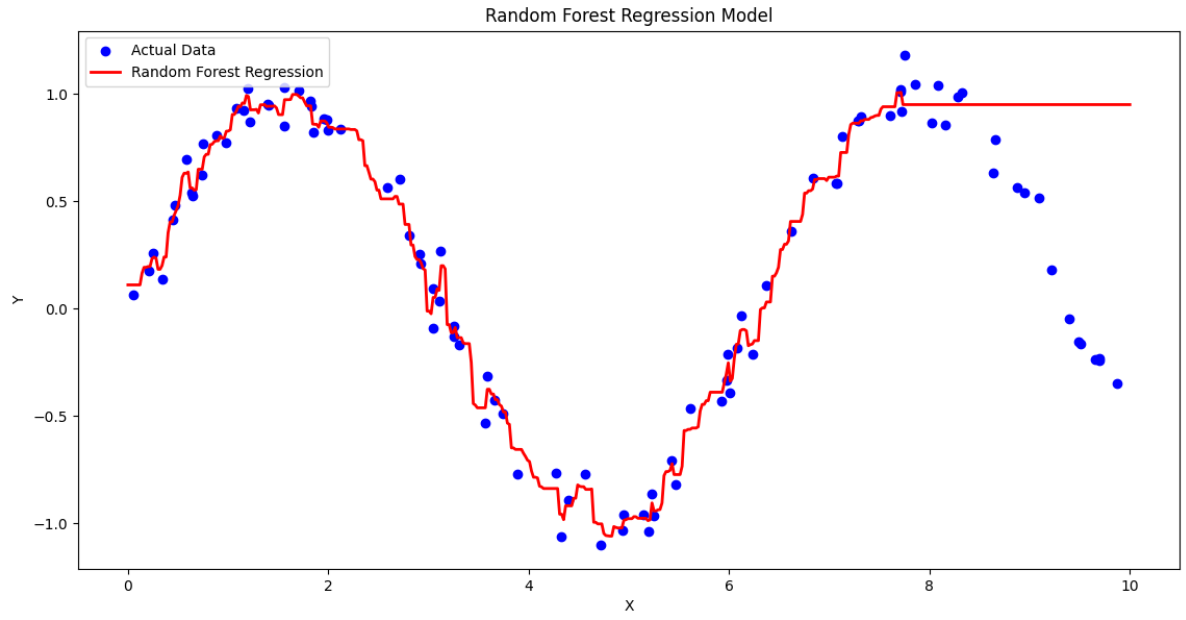
The code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
# Generating sample data
np.random.seed(42)
X = np.sort(10 * np.random.rand(100, 1), axis=0)  # Random X values
```

```
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])  # Non-linear function with
noise
# Splitting data into training and testing sets
train_size = int(0.8 * len(X))
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
# Creating and training the Random Forest Regressor
rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)
rf_regressor.fit(X_train, y_train)
# Making predictions
y_pred_train = rf_regressor.predict(X_train)
y_pred_test = rf_regressor.predict(X_test)
# Evaluating the model
train_mse = mean_squared_error(y_train, y_pred_train)
test_mse = mean_squared_error(y_test, y_pred_test)
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)
# Printing evaluation metrics
print(f"Train MSE: {train_mse:.4f}, Train R2: {train_r2:.4f}")
print(f"Test MSE: {test_mse:.4f}, Test R2: {test_r2:.4f}")
# Plotting the results
X_grid = np.linspace(0, 10, 500).reshape(-1, 1)  # More points for smoother curve
y_pred_grid = rf_regressor.predict(X_grid)
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(X_grid, y_pred_grid, color="red", linewidth=2, label="Random Forest Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Random Forest Regression Model")
plt.legend()
plt.show()
```

The results (Screenshot)

```
Train MSE: 0.0022, Train R2: 0.9951
Test MSE: 0.5349, Test R2: -0.9588
```



Conclusion

In conclusion, In regression problems, modeling appropriately selected model with training on labeled data for optimizing all its parameters for reducing error. The trained model is evaluated using different metrics such as MSE or R^2. Finally, the model will be used for prediction on new data with ensured accuracy and reliability.